Name:

Points: /20

# Databasutveckling och lagring — Final Exam: RETAKE II

2025-03-06

## Instructions

1. **No computers, no phones, no notes**
2. **No talking with other people in the room**
3. In case point 1 or 2 are broken, you get automatically **0 points** on the test.
4. Write your name on top of the page (do not fill in the *Points* box!)
5. For each question, you'll need to answer the question by crossing with an **X** the right answer (multiple choice questions) or write your answer in the empty lines below the question (open questions).
6. Once you're done:
   - Double-check your answers
   - Hand over your test to me
   - Leave the room

**NB:** While inside the classroom, rule 1 and 2 are still valid, even if you've already handed over your answers to me. If you want to discuss your answers with a classmate, or to check an answer on your phone, do that **outside the classroom**.

**NB:** Make sure to use the restroom, fill our water bottle, grab a coffee, etc... **before** we begin the exam, as you're not allowed to leave the classroom without handing over your answers.

## Point system

There are **10 questions**, worth **2 points** each, for a total of maximum **20 points**.

## There are no partial points

You either give the **complete, correct answer**, and get 2 points, or get 0 points.

## Grading

| Points | Grade |
|---|---|
| 0 - 12 | IG |
| 13 - 19 | G |
| 20 | VG |

READ EVERYTHING CAREFULLY!

# 1 Normalization

Given the following schema of a SQLite database:

Listing 1: The complete schema for the DB

```
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS customer (
        customer_id INTEGER PRIMARY KEY,
        full_name VARCHAR(100) NOT NULL,
        email VARCHAR(100) UNIQUE NOT NULL,
        postal_code VARCHAR(5) NOT NULL,
        city VARCHAR(50) NOT NULL
);

CREATE TABLE IF NOT EXISTS orders (
        order_id INTEGER PRIMARY KEY,
        order_date TEXT NOT NULL,
        customer_id INTEGER NOT NULL,
        FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
);

CREATE TABLE IF NOT EXISTS products (
        product_id INTEGER PRIMARY KEY,
        product_name TEXT NOT NULL,
        category TEXT,
        unit_price FLOAT NOT NULL
);

CREATE TABLE IF NOT EXISTS order_items (
        order_id INTEGER NOT NULL,
        product_id INTEGER NOT NULL,
        quantity INTEGER NOT NULL,
        product_name TEXT NOT NULL,
        PRIMARY KEY (order_id, product_id),
        FOREIGN KEY (order_id) REFERENCES orders(order_id),
        FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

Point out and explain **one** normalization problem present in the schema.

order_id is added as a Primary Key in orders table and then again in order_items table.

**2** **Explain how you would solve the normalization problem you described above.**

You don't need to write a SQL query, just explain how you would fix it.

I would remove the order_id primary key in the order_items table and keep the FOREIGN KEY.

**3** **In this course we've labbed with a database that contained the data for a music store. What was the name of the database, and which RDBMS did it use?**

Name: __________

RDBMS: __________

**4** **In this course, we've labbed with a database that contained the data for a movie rental store. What was the name of the database, and which RDBMS did it use?**

Name: __________

RDBMS: __________

**2** **Explain how you would solve the normalization problem you described above.**

You don't need to write a SQL query, just explain how you would fix it.

I Would remove the order_id Primary key in the order_items table and keep the FOREIGN KEY.

**3** **In this course we've labbed with a database that contained the data for a music store. What was the name of the database, and which RDBMS did it use?**

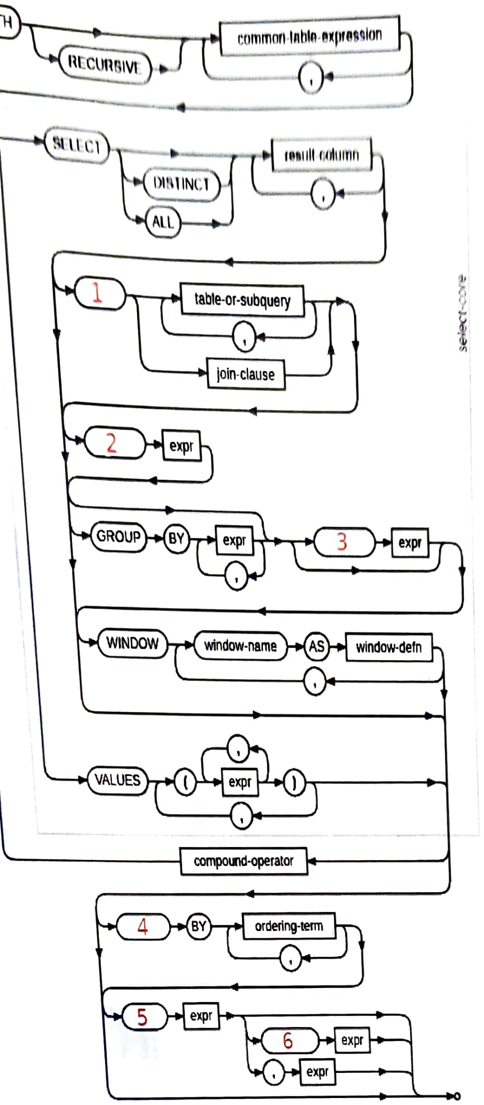
Name: __________

RDBMS: __________

**4** **In this course, we've labbed with a database that contained the data for a movie rental store. What was the name of the database, and which RDBMS did it use?**

Name: __________

RDBMS: __________


TH
RECURSIVE
common-table-expression
,
SELECT
DISTINCT
ALL
result-column
,
1
table-or-subquery
,
join-clause
2
expr
GROUP
BY
expr
,
3
expr
WINDOW
window-name
AS
window-defn
,
VALUES
(
expr
,
)
,
select-core
compound-operator
4
BY
ordering-term
,
5
expr
6
expr
,
expr

**6** What's the difference between `WHERE` and `HAVING` in a `SELECT` query?

With WHERE You can set conditions example: You can ask to Show Products with a Price below 100. With HAVING You can narrow your query, example: Searching for a Product Beginning with a specific letter.

**7** What's the difference between '`JOIN...USING`' and '`JOIN...ON`'?

**8** We can always replace '`JOIN...ON`' with '`JOIN...USING`'

1. True
2. False

**9** We can always replace '`JOIN...USING`' with '`JOIN...ON`'

1. True
2. False

**10 What is a subquery?**